

Java集合框架



2024/11/18

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

主要内容

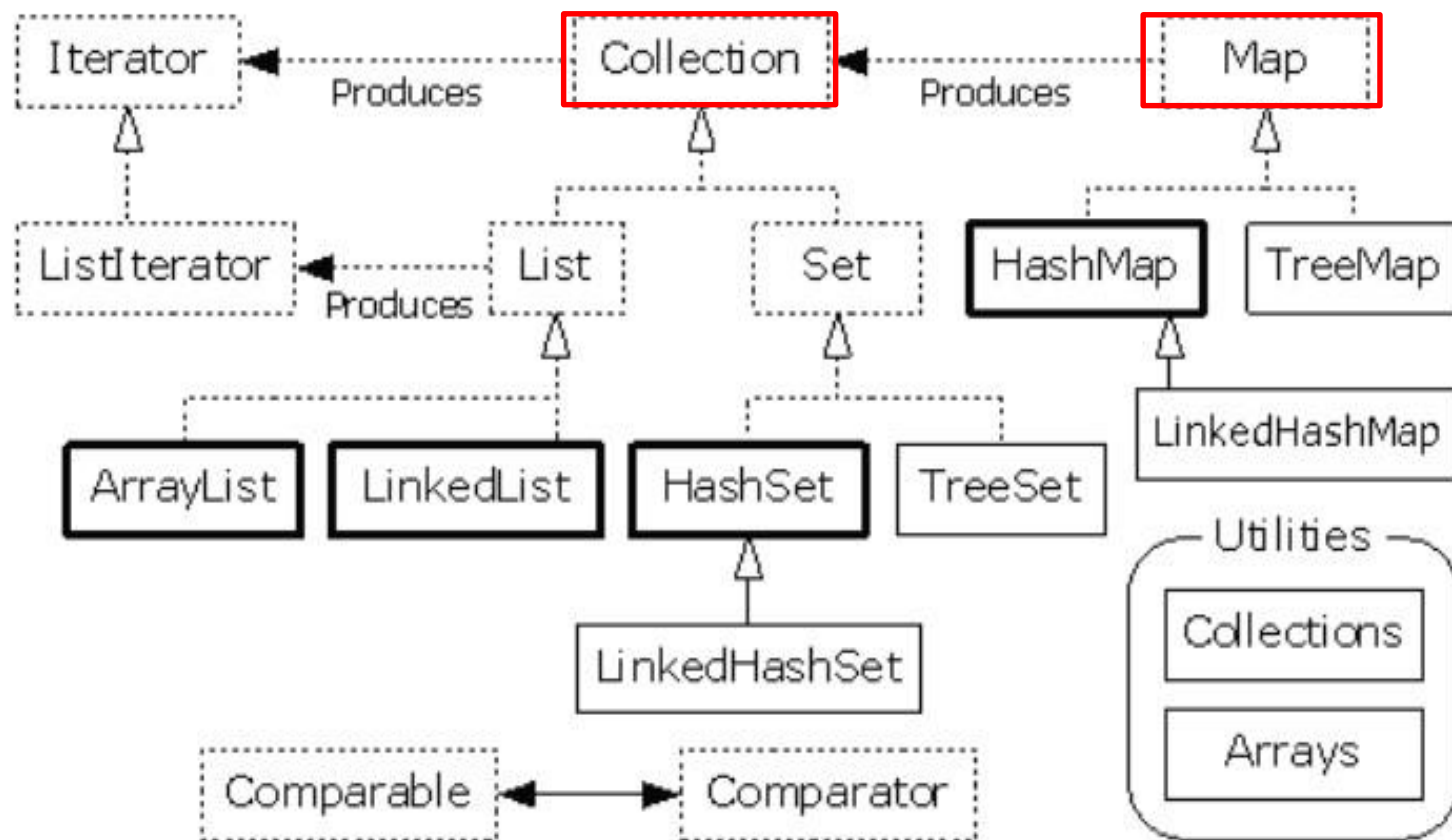
- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

- Java 集合框架中主要封装的是典型的数据结构和算法，如动态数组、双向链表、队列、栈、Set、Map 等
 - Java 集合就像一种容器，可以把多个对象的引用放入容器中。
 - Java 集合类可以用于存储数量不等的多个对象，还可用于保存具有映射关系的关联数组。

- **Java集合框架 (Java collection framework)** 方便对数据的存储与管理。
- 集合框架的特点
 - 高性能。基本集合（动态数组，链表，树，哈希表）的实现也必须是高效的。
 - 该框架允许不同类型的集合，以类似的方式工作，具有高度的互操作性。
 - 对一个集合的扩展和适应必须是简单的。

Java中集合结构图

简化图:

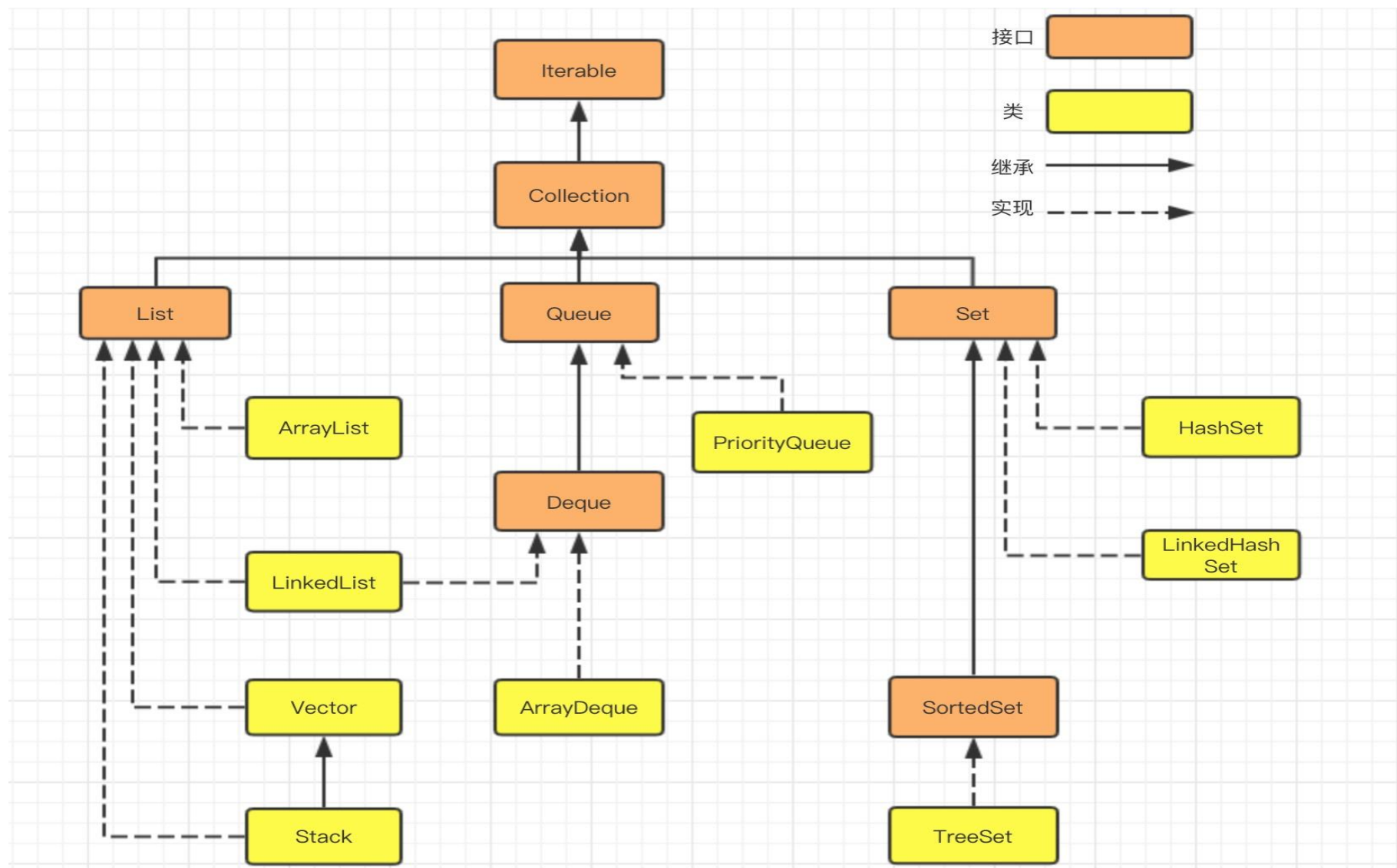


Java集合概述

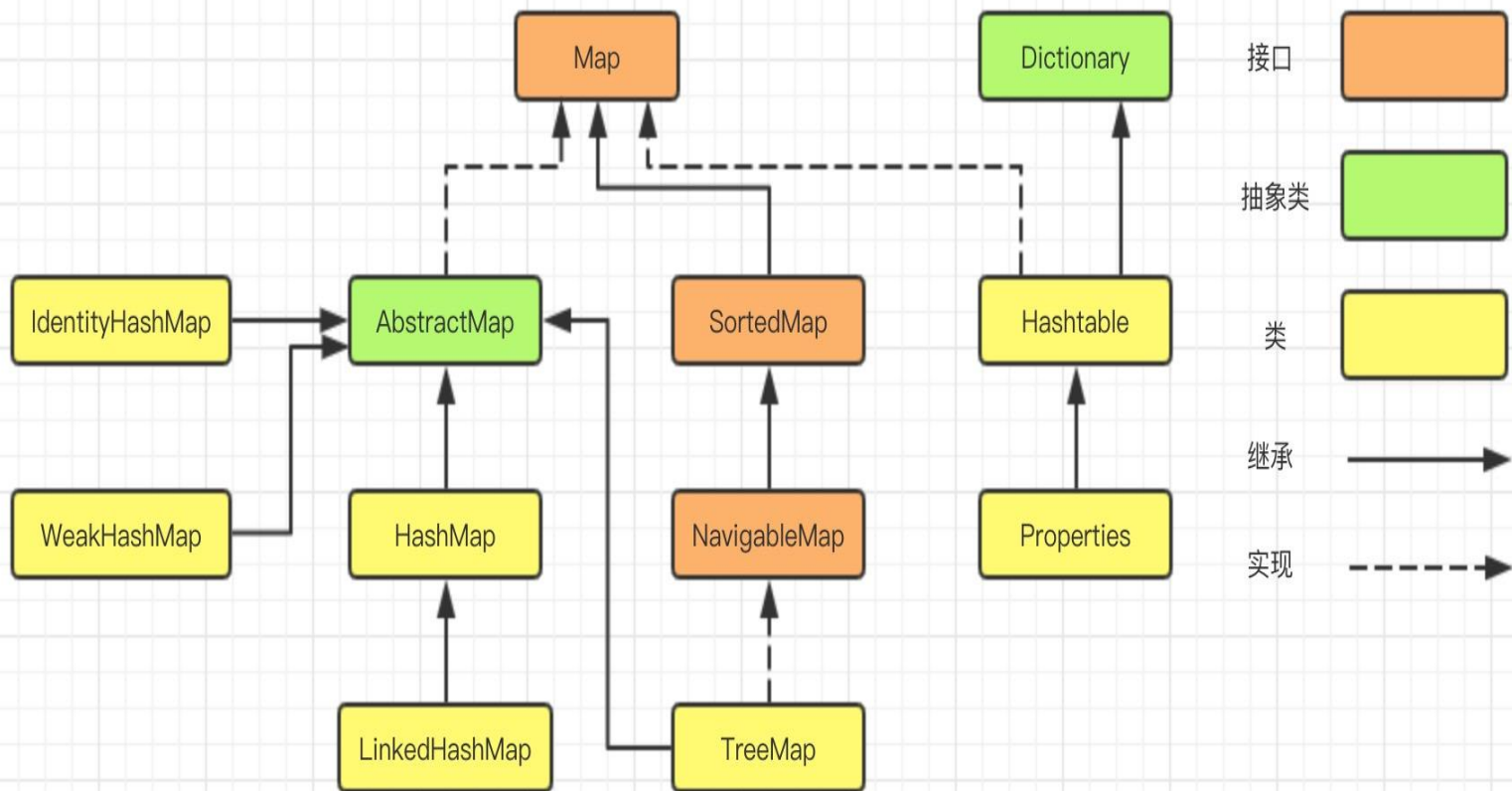
1. 所有集合类都位于 java.util 包下。Java的集合类主要由 **Collection** 和 **Map** 派生来；
2. Collection 和 Map 是 Java 集合框架的根接口，这两个接口又包含了一些子接口或实现类。
 - **集合接口**：短虚线表示，表示不同集合类型，是集合框架的基础。
 - **抽象类**：对集合接口的部分实现。可扩展为自定义集合类。
 - **实现类**：实线表示，对接口的具体实现。

- Collection 接口
 - **List** 列表是有序集合，集合中的元素可以重复，访问集合中的元素可以根据元素的索引来访问。
 - **Set** 集合是无序集合，集合中的元素不可以重复，访问集合中的元素只能根据元素本身来访问（也是集合里元素不允许重复的原因）。
- Map接口是键-值对象，与Collection接口没有什么关系。
 - Map 集合中保存 Key-value 对形式的元素，访问时只能根据每项元素的 key 来访问其 value。

java.util.Collection下的接口和继承类关系简易结构图:



java.util.Map下的接口和继承类关系简易结构图

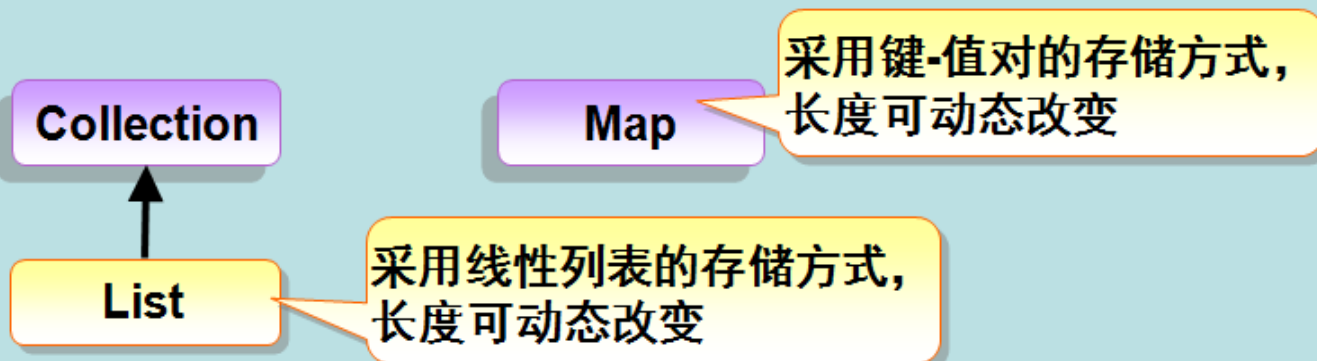


将集合框架挖掘处理，可以分为以下几个部分

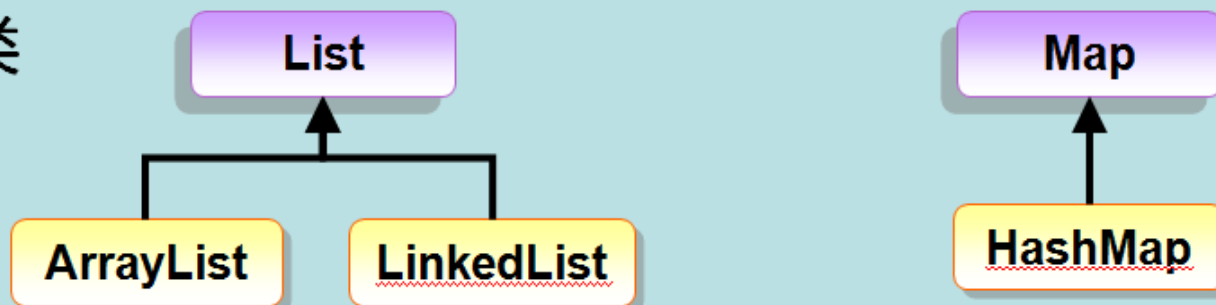
1. **数据结构**：List列表、Queue队列、Deque双端队列、Set集合、Map映射
2. **比较器**：Comparator比较器和Comparable排序接口
3. **算法**：Collections常用算法类、Arrays静态数组的排序、查找算法
4. **迭代器**：Iterator通用迭代器、ListIterator针对List特化的迭代器

Java集合框架包含的内容：接口、具体类和算法

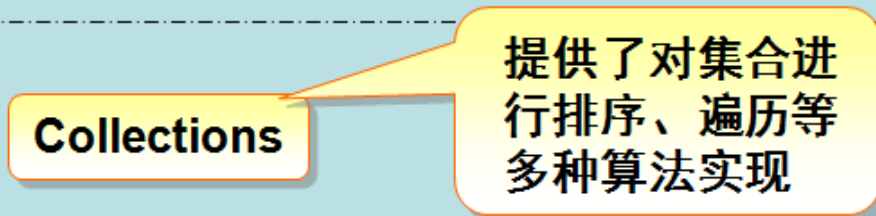
1 接口



2 具体类



3 算法



主要内容

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

集合类特点

- **只容纳对象**。这一点和数组不同，数组可以容纳基本数据类型数据和对象。
 - 如果集合类中想使用基本数据类型，又想利用集合类的灵活性，**可以把基本数据类型数据封装成该数据类型的对象**，然后放入集合中处理。
 - 集合类容纳的对象都是Object类的实例，一旦把一个对象置入集合类中，它的类信息将丢失，这样设计的目的是为了集合类的通用性。
 - 因为Object类是所有类的祖先，所以可以在这些集合中存放任何类的对象而不受限制，但是**切记在使用集合成员之前必须对它重新造型**。

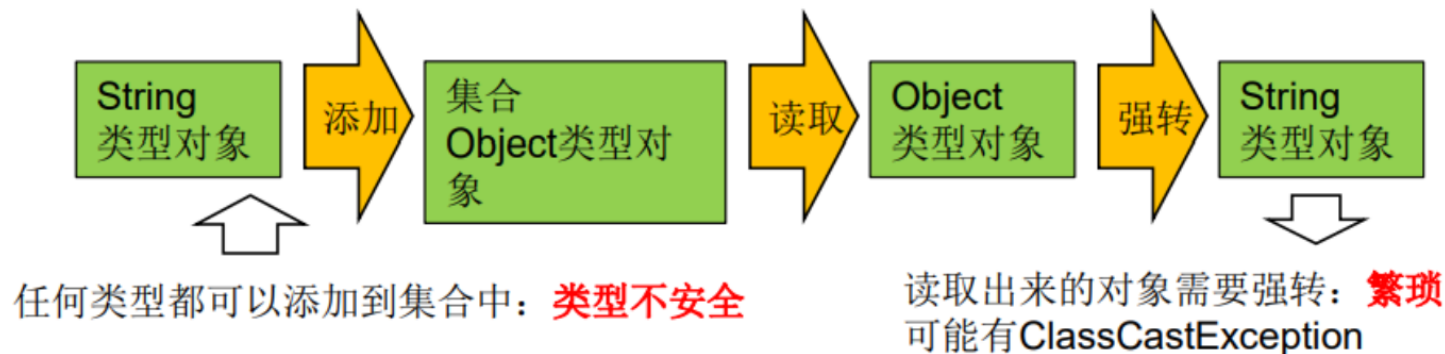
泛型集合的必要性

- 把任何类型对象通过`add(Object obj)`放入List中，认为只是Object类型
- 通过`get(int index)`取出List中元素时必须进行强制类型转换，繁琐而且容易出现异常
- 使用Map的`put(Object key, Object value)`和`get(Object key)`存取对象时存在同样问题
- 使用Iterator的`next()`方法获取元素时存在同样问题

那么为什么要有泛型呢，直接**Object**不是也可以存储数据吗？

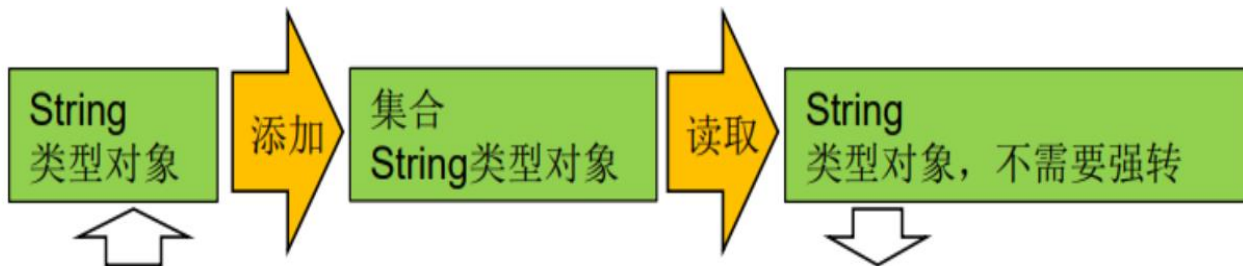
1. 解决元素存储的安全性问题，好比商品、药品标签，不会弄错。
2. 解决获取数据元素时，需要类型强制转换的问题，好比不用每回拿商品、药品都要辨别。

在集合中没有泛型时



```
List list = new ArrayList();  
list.add(e: "string");  
String d = (String) list.get(index: 0);
```

在集合中有泛型时



只有指定类型才可以添加到集合中：**类型安全** 读取出来的对象不需要强转：**便捷**

Java泛型可以保证如果程序在编译时没有发出警告，运行时就不会产生ClassCastException异常。同时，代码更加简洁、健壮。

```
List<String> list2 = new ArrayList<String>();  
list2.add(e: "string");  
String e = list2.get(index: 0);
```


泛型的高级应用：

```
public class GenericsFoo<T> {  
    private T x;  
    public GenericsFoo(T x) {  
        this.x = x;  
    }  
    public T getX() {  
        return x;  
    }  
    public void setX(T x) {  
        this.x = x;  
    }  
}
```

泛型扩展

```
public static void main(String args[]){  
    GenericsFoo<String> strFoo=new GenericsFoo<String>("Hello  
Generics!");  
    GenericsFoo<Double> douFoo=new GenericsFoo<Double>(new  
Double("33"));  
    GenericsFoo<Object> objFoo=new GenericsFoo<Object>(new  
Object());  
  
    System.out.println("strFoo.getX="+strFoo.getX());  
    System.out.println("douFoo.getX="+douFoo.getX());  
    System.out.println("objFoo.getX="+objFoo.getX());  
}  
}
```

限制泛型可用类型

在上面的例子中，由于没有限制class GenericsFoo<T>类型持有者T的范围，实际上这里的限定类型相当于Object，这和“Object泛型”实质是一样的。限制比如我们要限制T为集合接口类型。只需要这么做：

class GenericsFoo<T extends Collection>，这样类中的泛型T只能是Collection接口的实现类，传入非Collection接口编译会出错。

注意：<T extends Collection>这里的限定使用关键字extends，后面可以是类也可以是接口。但这里的extends已经不是继承的含义了，应该理解为T类型是实现Collection接口的类型，或者T是继承了XX类的类型。

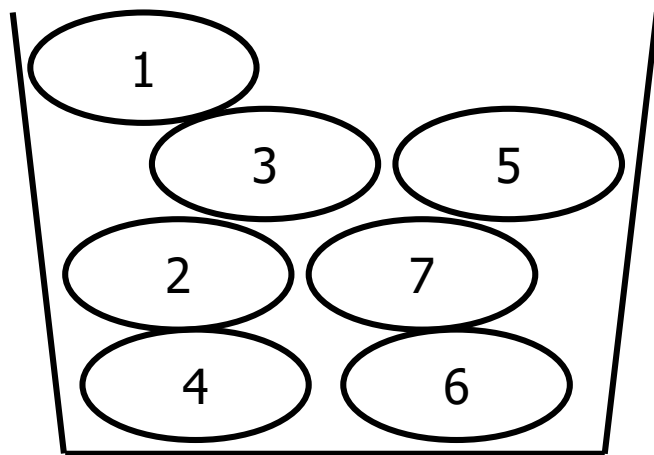
主要内容

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

- Collection 接口是 List、Set 和 Queue 接口的父接口，该接口里定义的方法既可用于操作 Set 集合，也可用于操作 List

Set

- Set中的元素必须唯一。
- 添加到Set中的元素必须定义equals方法，以提供算法来判断欲添加进来的对象是否与已经存在的某对象相等，从而建立对象的唯一性。
- 实现Set接口的类有HashSet, TreeSet

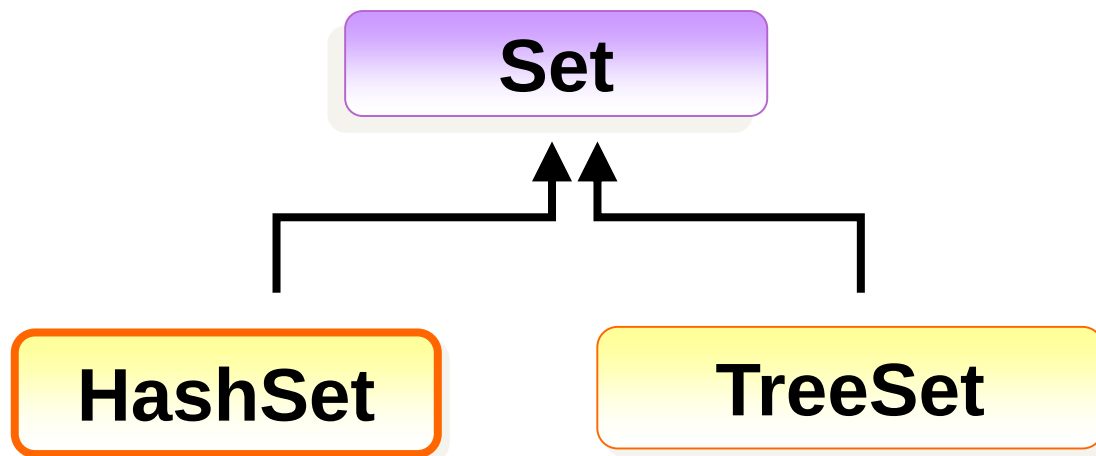


Set举例

```
public class SetDemo {  
    public static void main(String[] argv) {  
        HashSet<String> h = new HashSet<String>();  
        //也可以 Set h=new HashSet()  
        h.add("One");  
        h.add("Two");  
        h.add("One"); // DUPLICATE  
        h.add("Three");  
        Iterator<String> it = h.iterator();  
        while (it.hasNext()) {  
            System.out.println(it.next());  
        }  
    }  
}
```

Three
One
Two

Set 分类



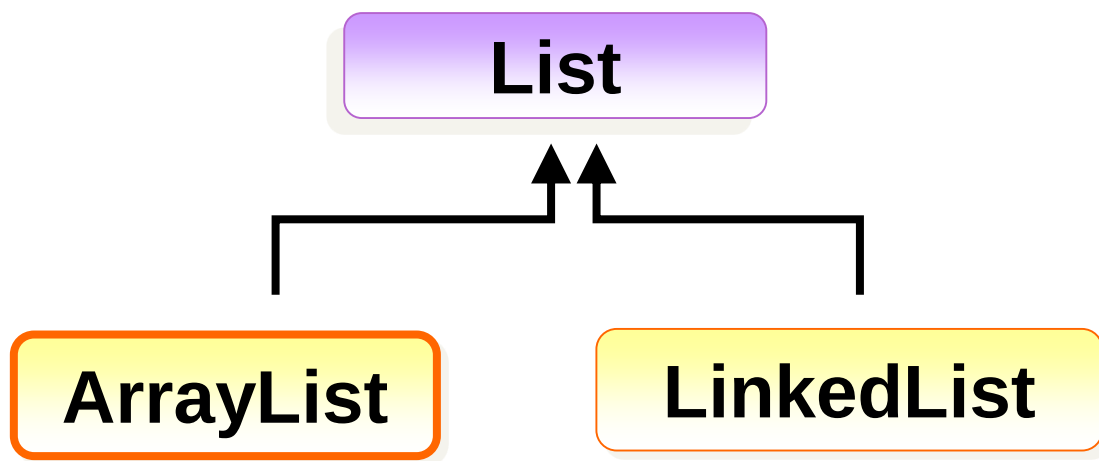
HashSet vs TreeSet

- TreeSet 是二差树实现的,TreeSet中的数据是自动排好序的,不允许放入null值。
- HashSet 是哈希表实现的,HashSet中的数据是无序的,可以放入null,但只能放入一个null,两者中的值都不能重复,就如数据库中唯一约束。
- HashSet要求放入的对象必须实现HashCode()方法,放入的对象,是以hashcode码作为标识的。
 - 具有相同内容的String对象,hashcode是一样,所以放入的内容不能重复。但是同一个类的对象可以放入不同的实例。

主要内容

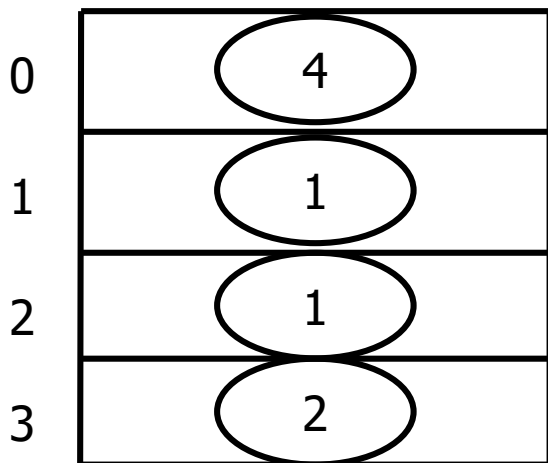
- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

List接口实现类



List

- List的明显特征是它的元素都有一个确定的顺序。
- 实现它的类有ArrayList和LinkedList。
 - ArrayList内存中是顺序存储的。
 - LinkedList内存中是以链表方式存储的



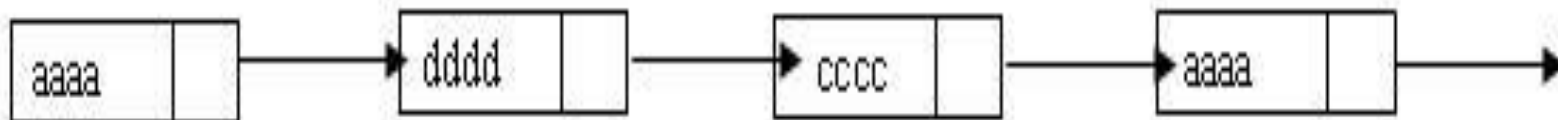
List (列表)

List接口实现类

- **ArrayList**实现了长度可变的数组，在内存中分配连续的空间。遍历元素和随机访问元素的效率比较高

0	1	2	3	4	5	
aaaa	dddd	cccc	aaaa	eeee	dddd	

- **LinkedList**采用链表存储方式。插入、删除元素时效率比较高



ArrayList集合类

- 存储多条狗狗信息，获取狗狗总数，逐条打印出各条狗狗信息
- 通过List接口的实现类ArrayList实现该需求
 - 元素个数不确定
 - 要求获得元素的实际个数
 - 按照存储顺序获取并打印元素信息

```
import java.util.*;
public class Test1 {
    public static void main(String[] args) {
        Dog ououDog = new Dog("欧欧", "雪娜瑞");
        Dog yayaDog = new Dog("亚亚", "拉布拉多");
        Dog meimeiDog = new Dog("美美", "雪娜瑞");
        Dog feifeiDog = new Dog("菲菲", "拉布拉多");

        List<Dog> dogs = new ArrayList<Dog>(); // 泛型
        dogs.add(ououDog);
        dogs.add(yayaDog);
        dogs.add(meimeiDog);
        dogs.add(2, feifeiDog); // 添加feifeiDog到指定位置

        System.out.println("共计有" + dogs.size() + "条狗狗。");

        System.out.println("分别是: ");
        for (int i = 0; i < dogs.size(); i++) {
            Dog dog = (Dog) dogs.get(i);
            System.out.println(dog.getName() + "\t" + dog.getStrain());
        }
    }
}
```

ArrayList接口常用方法

方法名	说 明
<code>boolean add(Object o)</code>	在列表的末尾顺序添加元素，起始索引位置从0开始
<code>void add(int index, Object o)</code>	在指定的索引位置添加元素。索引位置必须介于0和列表中元素个数之间
<code>int size()</code>	返回列表中的元素个数
<code>Object get(int index)</code>	返回指定索引位置处的元素。取出的元素是Object类型，使用前需要进行强制类型转换
<code>boolean contains(Object o)</code>	判断列表中是否存在指定元素
<code>boolean remove(Object o)</code>	从列表中删除元素
<code>Object remove(int index)</code>	从列表中删除指定位置元素，起始索引位置从0开始

LinkedList集合类

- 在集合任何位置（头部、中间、尾部）添加、获取、删除狗狗对象
- 插入、删除操作频繁时，可使用LinkedList来提高效率
- LinkedList还额外提供对头部和尾部元素进行添加和删除操作的方法

示例代码

```
public class Test3 {  
    public static void main(String[] args) {  
        Dog ououDog = new Dog("欧欧", "雪娜瑞");  
        Dog yayaDog = new Dog("亚亚", "拉布拉多");  
        Dog meimeiDog = new Dog("美美", "雪娜瑞");  
        Dog feifeiDog = new Dog("菲菲", "拉布拉多");  
  
        LinkedList<Object> dogs = new LinkedList<Object>();  
        // LinkedList<Dog> dogs = new LinkedList<Dog>();  
        dogs.add(ououDog);  
        dogs.add(yayaDog);  
        dogs.addFirst(meimeiDog); // 添加meimeiDog到指定位置  
        dogs.addLast(feifeiDog); // 添加feifeiDog到指定位置  
  
        System.out.println("共计有" + dogs.size() + "条狗狗。");  
  
        System.out.println("分别是: ");  
        for (int i = 0; i < dogs.size(); i++) {  
            Dog dog = (Dog) dogs.get(i);  
            System.out.println(dog.getName() + "\t" + dog.getStrain());  
        }  
    }  
}
```

```
Dog dogFirst= (Dog)dogs.getFirst();
System.out.println("第一条狗狗昵称是"+dogFirst.getName() );

Dog dogLast= (Dog)dogs.getLast();
System.out.println("最后一条狗狗昵称是"+dogLast.getName());

dogs.removeFirst();
dogs.removeLast();
System.out.println("共计有" + dogs.size() + "条狗狗。");

System.out.println("分别是: ");
for (int i = 0; i < dogs.size(); i++) {
    Dog dog = (Dog) dogs.get(i);
    System.out.println(dog.getName() + "\t" + dog.getStrain());
}
```

LinkedList的特殊方法

方法名	说 明
<code>void addFirst(Object o)</code>	在列表的首部添加元素
<code>void addLast(Object o)</code>	在列表的末尾添加元素
<code>Object getFirst()</code>	返回列表中的第一个元素
<code>Object getLast()</code>	返回列表中的最后一个元素
<code>Object removeFirst()</code>	删除并返回列表中的第一个元素
<code>Object removeLast()</code>	删除并返回列表中的最后一个元素

主要内容

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

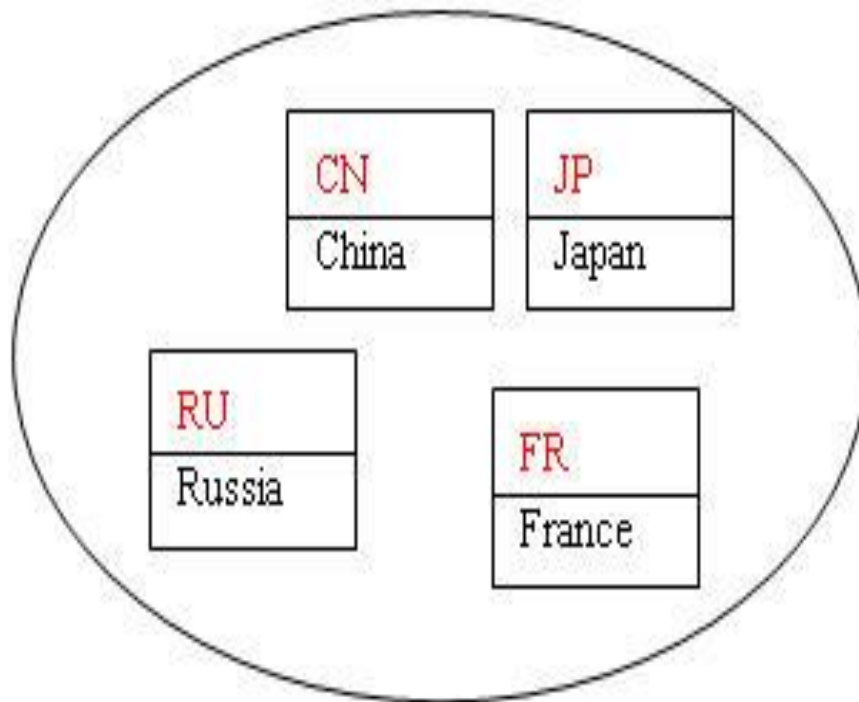
Map接口

- 建立国家英文简称和中文全名间的键值映射，并通过key对value进行操作，应该如何实现数据的存储和操作呢？
- Map接口专门处理键值映射数据的存储，可以根据键实现对值的操作
- **最常用的实现类是HashMap**

Java集合框架包含的内容（2）

- Map接口存储一组键值对象，提供key到value的映射

Map



```

public class Test4 {
    public static void main(String[] args) {
        Map<String,String> countries =
            new HashMap<String,String>();
        countries.put("CN", "中华人民共和国");
        countries.put("RU", "俄罗斯联邦");
        countries.put("FR", "法兰西共和国");
        countries.put("US", "美利坚合众国");
        String country = (String) countries.get("CN");
        System.out.println("CN对应的国家是: " + country);
        System.out.println("Map中共有"+countries.size()+"组数据");
        countries.remove("FR");
        System.out.println("Map中包含FR的key吗? " +
            countries.containsKey("FR"));
        System.out.println( countries.keySet() );
        System.out.println( countries.values() );
        System.out.println( countries );
    }
}

```


输出结果

CN对应的国家是：中华人民共和国

Map中共有**4**组数据

Map中包含**FR**的**key**吗？ **false**

[US, RU, CN]

[美利坚合众国, 俄罗斯联邦, 中华人民共和国]

{US=美利坚合众国, RU=俄罗斯联邦, CN=中华人民共和国}

Map接口常用方法

方法名	说 明
Object put(Object key, Object val)	以“键-值对”的方式进行存储
Object get (Object key)	根据键返回相关联的值，如果不存在指定的键，返回null
Object remove (Object key)	删除由指定的键映射的“键-值对”
int size()	返回元素个数
Set keySet ()	返回键的集合
Collection values ()	返回值的集合
Boolean containsKey (Object key)	如果存在由指定的键映射的“键-值对”，返回true

Hashtable和HashMap

- Hashtable和HashMap的异同
 - 实现原理相同，功能相同，在很多情况下可以互用
 - 两者的主要区别如下
 - Hashtable继承Dictionary类，HashMap实现Map接口
 - Hashtable线程安全，HashMap线程不安全
 - Hashtable不允许null值，HashMap允许null值

主要内容

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

如何遍历List集合

- 方法1：通过for循环和get()方法配合实现遍历
- 方法2：增强型for循环
- 方法3：通过迭代器Iterator实现遍历

迭代器模式

- 迭代器模式提供一种方法顺序访问一个聚合对象中的各个元素，而又不暴露其内部的表示。
 - **迭代器 (Iterator)**：迭代器定义访问和遍历元素的接口。
 - **具体迭代器 (ConcreteIterator)**：具体迭代器实现迭代器接口，对该聚合遍历时跟踪当前位置。
 - **聚合 (Aggregate)**：聚合定义创建相应迭代器对象的接口。
 - **具体聚合 (ConcreteAggregate)**：具体聚合实现创建相应迭代器的接口，该操作返回 ConcreteIterator 的一个适当的实例。

迭代器模式

- 包子店与沙县小吃合并啦：有一家包子店，他们负责提供早餐，卖各式各样的包子；也有一家沙县小吃，它们负责提供午餐和晚餐，卖一些盖浇饭和煲仔饭。为了获得更多的盈利，后来这两家店合并了。
- 但是却发现了一个问题，原来的包子店提供的菜单是用 List 集合结合存储的，而沙县小吃的菜单却是由数组存储的。所以在提供菜单的时候就会很麻烦，需要使用两个不同的循环来遍历对应的集合和数组，才能打印完整的菜单。
- 有没有更好的解决办法呢？这时候迭代器模式就登场了。

抽象聚合类和具体聚合类

```
import java.util.Iterator;
```

```
public interface Menu {  
    Iterator<MenuItem> createIterator();  
}
```

```
public BZMenu() {  
    menuItems = new ArrayList<>();  
  
    addItem("雪菜包", 1.0);  
    addItem("灌汤包", 1.0);  
    addItem("韭菜包", 1.0);  
    addItem("猪肉包", 1.0);  
}  
  
@Override  
public Iterator<MenuItem> createIterator() {  
    return new BZMenuIterator(menuItems);  
}
```


具体迭代器BZMenuItemIterator.java

```
public class BZMenuItemIterator implements Iterator {
    List<MenuItem> menuItems;
    int position = 0;

    @Override
    public boolean hasNext() {
        if(position >= menuItems.size() || menuItems.get(position) == null){
            return false;
        }else {
            return true;
        }
    }

    @Override
    public Object next() {
        MenuItem menuItem = menuItems.get(position);
        position += 1;
        return menuItem;
    }
}
```

使用迭代器

```
public Waitress(BZMenu bzMenu, SXMenu sxMenu){
    this.bzMenu = bzMenu;
    this.sxMenu = sxMenu;
}

public void printMenu(){
    Iterator<MenuItem> iterator = bzMenu.createIterator();
    Iterator<MenuItem> iterator1 = sxMenu.createIterator();

    System.out.println("包子菜单为: ");
    printMenu(iterator);
    System.out.println("沙县菜单为: ");
    printMenu(iterator1);
}

private void printMenu(Iterator iterator){
    while (iterator.hasNext()){
        MenuItem menuItem = (MenuItem) iterator.next();
        System.out.print(menuItem.getName() + ": ");
        System.out.println(menuItem.getPrice());
    }
}
```

迭代器的使用场景

- 访问一个聚合对象的内容而无需暴露它的内部表示。
- 支持对聚合对象的多种遍历。
- 为遍历不同的聚合结构提供一个统一的接口。

通过迭代器Iterator实现遍历

- 所有集合接口和类都没有提供相应遍历方法，而是由Iterator实现集合遍历
- Collection 接口的iterate()方法返回一个Iterator，然后通过Iterator接口的两个方法可实现遍历
 - **boolean hasNext():** 判断是否存在另一个可访问的元素
 - **Object next():** 返回要访问的下一个元素

使用 Iterator 接口遍历集合元素

- Iterator 接口主要用于遍历 Collection 集合中的元素，Iterator 对象也被称为迭代器
- Iterator 接口隐藏了各种 Collection 实现类的底层细节，向应用程序提供了遍历 Collection 集合元素的统一编程接口
- **Iterator 仅用于遍历集合，Iterator 本身并不提供承装对象的能力。如果需要创建 Iterator 对象，则必须有一个被迭代的集合。**

主要内容

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

Java中Collection和Collections的区别

- Collection是一个顶层接口，绝大多数常用的容器类都实现了Collection接口，里面定义的容器类的基本方法。
- Collections是一个工具类，此类不能实例化，服务于Java的Collection框架。他提供一系列静态方法实现对各种集合的搜索、排序、线程安全化等操作

Collection接口

Collection接口的意义：
是为各种具体的集合提供了最大化
的统一操作方式

```
Collection
```

- `add(E): boolean`
- `addAll(Collection<? extends E>): boolean`
- `clear(): void`
- `contains(Object): boolean`
- `containsAll(Collection<?>): boolean`
- `equals(Object): boolean` $\uparrow$ Object
- `forEach(Consumer<? super T>): void` $\rightarrow$ Iterable
- `hashCode(): int` $\uparrow$ Object
- `isEmpty(): boolean`
- `iterator(): Iterator<E>` $\uparrow$ Iterable
- `parallelStream(): Stream<E>`
- `remove(Object): boolean`
- `removeAll(Collection<?>): boolean`
- `removeIf(Predicate<? super E>): boolean`
- `retainAll(Collection<?>): boolean`
- `size(): int`
- `spliterator(): Spliterator<E>` $\uparrow$ Iterable
- `stream(): Stream<E>`
- `toArray(): Object[]`
- `toArray(T[]): T[]`

Collections是一个工具类，服务于Java的Collection框架

```
Collections
  Collections()
  addAll(Collection<? super T>, T...): boolean
  asLifoQueue(Deque<T>): Queue<T>
  binarySearch(List<? extends Comparable<? super T>>, T): int
  binarySearch(List<? extends T>, T, Comparator<? super T>): int
  checkedCollection(Collection<E>, Class<E>): Collection<E>
  checkedList(List<E>, Class<E>): List<E>
```

```
swap(Object[], int, int): void
synchronizedCollection(Collection<T>): Collection<T>
synchronizedCollection(Collection<T>, Object): Collection<T>
synchronizedList(List<T>): List<T>
synchronizedList(List<T>, Object): List<T>
synchronizedMap(Map<K, V>): Map<K, V>
synchronizedNavigableMap(NavigableMap<K, V>): NavigableMap<K, V>
synchronizedNavigableSet(NavigableSet<T>): NavigableSet<T>
synchronizedSet(Set<T>): Set<T>
synchronizedSet(Set<T>, Object): Set<T>
```

转换为线程安全的类

工具类Collections

- 工具类collections用于操作集合类，如List, Set, 常用方法有；
 - 1) 排序 (Sort)
 - 2) 混排 (Shuffling)
 - 3) 反转 (Reverse)
 - 4) 替换所有的元素 (Fill)
 - 5) 拷贝 (Copy)
 - 6) 返回Collections中最小元素 (min) 最大元素 (max)
 - 8) lastIndexOfSubList
 - 9) IndexOfSubList

工具类Collections

- 10) Rotate
- 11) static int binarySearch(List list, Object key) 使用二分搜索查找key对象的索引值，因为使用的二分查找，所以前提是必须有序。
- 12) static Object max(Collection coll) 根据元素自然顺序，返回集合中的最大元素
- 13) static Object max(Collection coll, Compare comp) 根据Comparator指定的顺序，返回给定集合中的最大元素
- 14) static Object min(Collection coll) 根据元素自然顺序，返回集合中的最小元素
- 15) static Object min(Collection coll, Compare comp) 根据Comparator指定的顺序，返回给定集合中的最小元素
- 16) static void fill(List list, Object obj) 使用指定元素替换指定集合中的所有元素
- 17) static int frequency(Collection c, Object o) 返回指定元素在集合中出现在次数
- 18) static int indexOfSubList(List source, List target) 返回子List对象在父List对象中第一次出现的位置索引；如果父List中没有出现这样的子List，则返回-1
- 19) static int lastIndexOfSubList(List source, List target) 返回子List对象在父List对象中最后一次出现的位置索引，如果父List中没有出现这样的子List，则返回-1
- 20) static boolean replaceAll(List list, Object oldVal, Object newVal) 使用一个新值newVal替换List对象所有旧值oldVal
- 21) synchronizedXXX(new XXX) Collections类为集合类们提供的同步控制方法



Collections.reverse

```
System.out.println(x: "LinkedList");
LinkedList<Student> stu3 = new LinkedList<Student>();
stu3.add(a);
stu3.add(b);
stu3.add(c);
stu3.add(index: 1, a);
for (Student stu : stu3) {
    System.out.println(stu.name + " -> " + stu.id);
}
```

```
System.out.println(x: "Collections reverse");
```

```
Collections.reverse(stu3);
```

```
for (Student stu : stu3) {
    System.out.println(stu.name + " -> " + stu.id);
}
```

LinkedList

李小明 -> 12345

李小明 -> 12345

李小明 -> 12346

王大朋 -> 12347

Collections reverse

王大朋 -> 12347

李小明 -> 12346

李小明 -> 12345

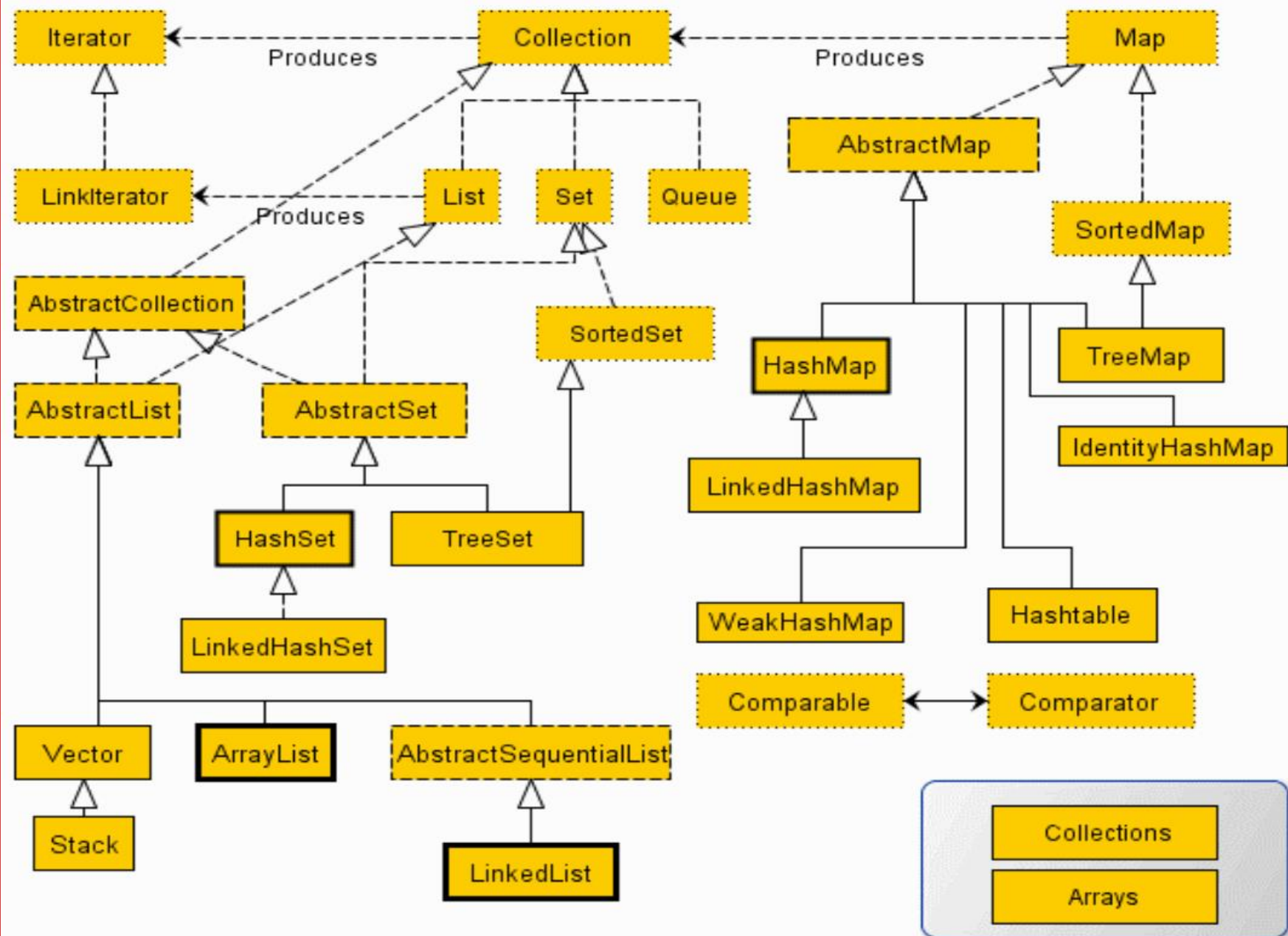
李小明 -> 12345

小结 (1)

- 集合就是将若干用途、性质形同或相近的数据组合而成的一个整体
- Java 集合可分为 **Set**、**List** 和 **Map** 三种体系
 - **Set**: 不区分元素的顺序, 不允许出现重复元素
 - **List**: 区分元素的顺序, 且允许包含重复元素
 - **Map**: 键-值 (Key-Value), 映射中保存成对的键-值信息, 映射中不能包含重复的键, 每个键最多只能映射一个值

小结 (2)

- Java 集合就像一种容器，可以把**多个对象的引用**放入容器中。
- Java 集合类可以用于存储数量不等的多个对象，还可用于保存具有映射关系的关联数组
- 在 Java5 之前，Java 集合会丢失容器中所有对象的数据类型，把所有对象都当成 Object 类型处理；从 **Java5 增加了泛型以后**，Java 集合可以记住容器中对象的数据类型



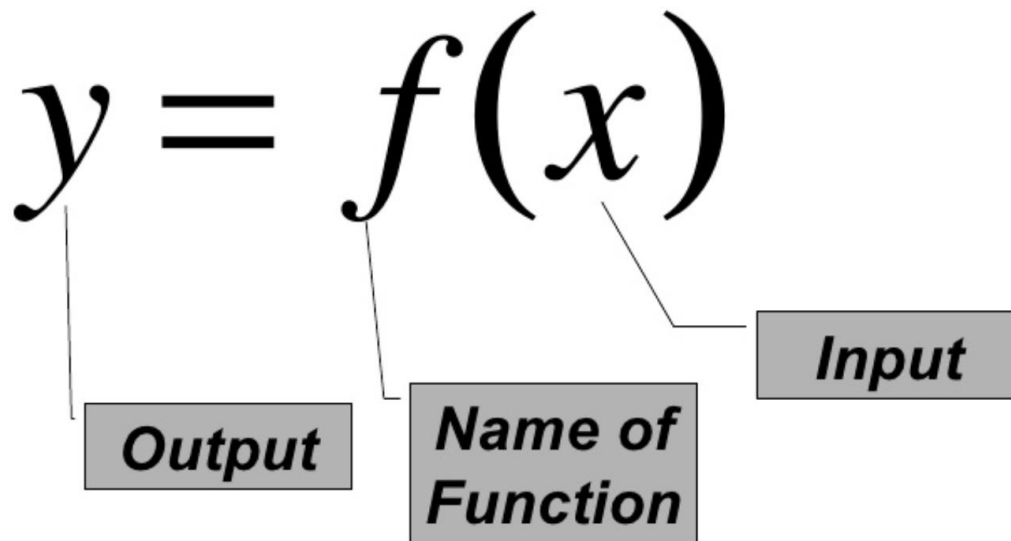
请回答以下问题

- Collection、List、Set、Map接口的联系和区别有哪些？
- ArrayList和LinkedList有什么异同之处？
- 有哪些遍历集合的方法？
- 为什么要引入泛型集合？

- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程

函数式编程(functional programming)

- 函数式编程是一种编程范式
- 函数式编程语言
 - Ocaml, Scala, Haskell, F#
 - Java 8+支持部分函数式编程



副作用 (Side effect)

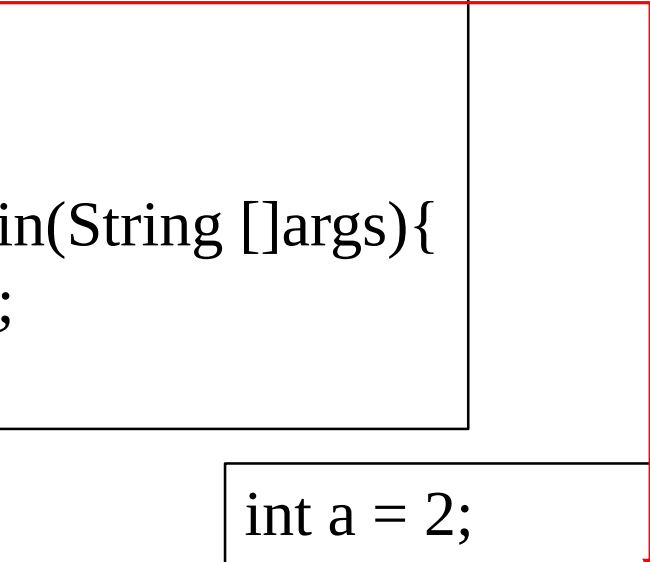
在函数式编程中，输入一旦确定了，输出都确定了，函数调用的结果只依赖于传入的输入变量和内部逻辑，不依赖于外部，这样的写出的函数没有副作用

```
int a = 2;  
public int add(int b){  
    return a + b;  
}  
  
public int pow(int c){  
    return c * c;  
}
```

Side effect

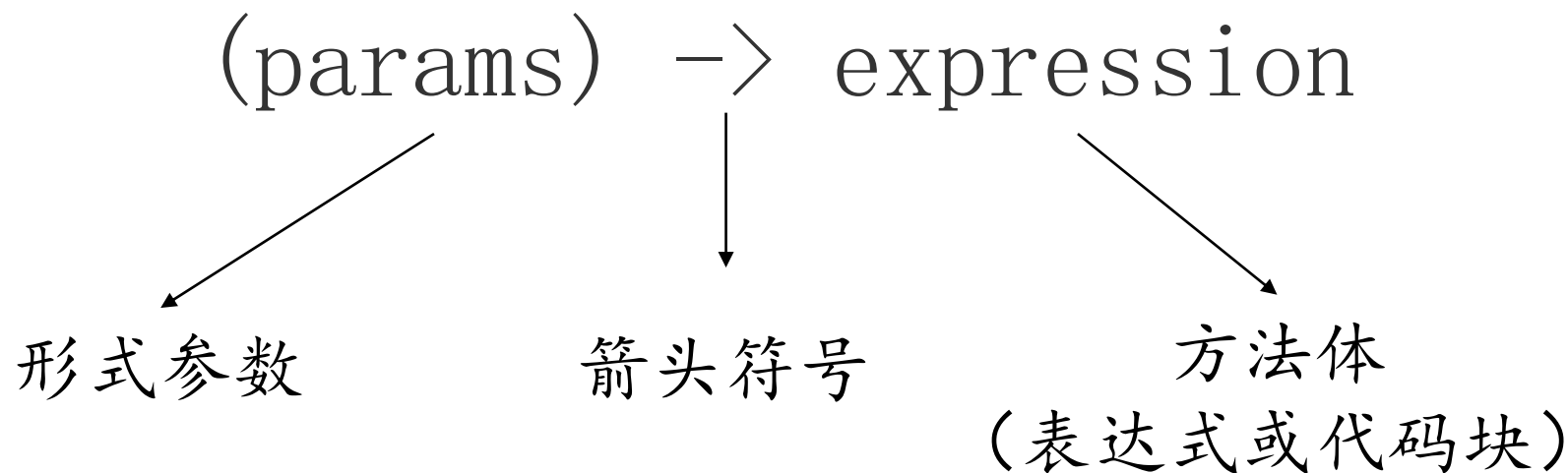
Make this method functional

```
int a = 2;  
public int add(int b){  
    return a + b;  
}  
Public static void main(String []args){  
    int c = add(3);  
}
```



```
int a = 2;  
public int add(int a, int b){  
    return a + b;  
}  
Public static void main(String []args){  
    int c = add(a, 3);  
}
```

lambda表达式



lambda表达式

```
BiFunction<Integer, Integer, Integer> f1 = (a, b) -> {  
    int c = a + b;  
    return c;  
};  
System.out.println("addition results: " + f1.apply(3, 2));  
  
BiFunction<Integer, Integer, Integer> f2 = (a, b) -> a * b;  
System.out.println("multiply results: " + f2.apply(3, 2));  
  
Function<Integer, Integer> f3 = a -> a * a;  
System.out.println("square results: " + f3.apply(3));
```

addition results: 5
multiply results: 6
square results: 9

lambda表达式

```
private static void calcuate(int i1, int i2,  
    BiFunction<Integer, Integer, Integer> f){  
    System.out.println("calculate results: " + f.apply(i1, i2));  
}
```

```
calcuate(i1: 2, i2: 3, f1);  
calcuate(i1: 2, i2: 3, f2);
```

calculate results: 5
calculate results: 6

lambda表达式

//数据的提供者, 只出不进。

```
Supplier<Integer> supplier = () -> new Random().nextInt();  
System.out.println("supplier: " + supplier.get());
```

//数据的消费者, 只进不出

```
Consumer<Integer> consumer = (Integer x) -> {  
    System.out.println("consumer value " + x);  
};  
consumer.accept(5);
```

//判断, 返回 true或者false。

```
Predicate<Integer> integerPredicate = x -> {  
    if (x > 23) {  
        return false;  
    }  
    return true;  
};  
integerPredicate.test(25);
```


使代码更简单

```
Thread t1 = new Thread(new Runnable(){  
    public void run(){  
        System.out.println(x: "使用匿名类的方式创建线程");  
    }  
});  
t1.start();
```



```
Thread t2 = new Thread(() -> System.out.println(x: "使用lambda方式创建线程"));  
t2.start();
```

Collection + Lambda

```
List<String> list = Arrays.asList("Tom", "Mary", "Huahua");
```

How to print the values of this list?

```
for(int i = 0; i<list.size(); i++){  
    System.out.println(list.get(i));  
}
```

```
for(String name: list){  
    System.out.println(name);  
}
```

```
list.forEach(name -> System.out.println(name));
```

Collection + Lambda

```
List<StudentInfo> studentList = new ArrayList<>();
```

How to filter the data?

```
System.out.println(x: "=====Without FP=====");
List<StudentInfo> tallBoys = new ArrayList<>();
for (StudentInfo stu : studentList) {
    if(stu.getGender()=='m' && stu.getHeight()>=1.7)
        tallBoys.add(stu);
}
for(StudentInfo boy: tallBoys)
    System.out.println(boy.toString());

System.out.println(x: "=====Using FP=====");
List<StudentInfo> boys = studentList.stream().filter(s -> s.getGender()
=='m' && s.getHeight() >= 1.7).collect(Collectors.toList());
boys.forEach(b->System.out.println(b));
```

Map + Lambda

```
Map<String, String> maps = new HashMap<String, String>();  
maps.put(key: "a", value: "A");  
maps.put(key: "b", value: "B");  
maps.put(key: "c", value: "C");
```

How to print the key and value of maps?

```
System.out.println(x: "=====Without FP=====");  
for(String key: maps.keySet()){  
    String value = maps.get(key);  
    System.out.println("key: " + key + ", value: " + value);  
}
```

```
System.out.println(x: "=====Using FP=====");  
maps.forEach((k, v) -> System.out.println("key: " + k + ", value: " + v));
```

Collection + Lambda

- `List<String> list = Arrays.asList("a1", "a2", "b1", "c2", "c1");`
- Find the elements starting with 'c'
- change 'c' to 'C'
- sorted
- print

Collection + Lambda

- `List<String> list = Arrays.asList("a1", "a2", "b1", "c2", "c1");`

```
list.stream().  
    filter(s->s.startsWith(prefix: "c"))  
    .map(String::toUpperCase)  
    .sorted()  
    .forEach(System.out::println);
```

Methods accepting Functions

- `java.util.Collections`
 - `sort`, `binarySearch`, `min`, `max`, etc
- `java.util.stream`
 - `map`, `reduce`, `filter`, `find`, `forEach`, `flatMap`, etc
- `Optional`
 - `ifPresent`, `orElse`, `filter`, `map`, etc
- Anything that accepts anonymous classes

Why Functional?

- Easy to write and understand



- Java 集合概述
 - Collection 接口
 - 泛型集合
 - Set
 - List
 - Map
 - 迭代器模式&Iterator 接口
 - 工具类collections
- 函数式编程